Synthetic Drifting Regression over Binary Feature Streams

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

This paper specifies a reproducible synthetic online regression task for binary feature vectors. Concept drift shows up across real-world ML, yet most frameworks still assume stationary train/serve data; this task takes nonstationarity as a first-class requirement with controllable drift axes rather than an afterthought. A synthetic setting makes it easy to experiment with controlled, reproducible scenarios—known latent mechanisms, seeded trajectories, and explicit scenario knobs—so adaptation can be studied without confounding from opaque real streams. The paper defines the synthetic environment, its independent sources of concept drift, the model boundary, and the prequential evaluation protocol. It is a task and library specification, not a performance report.

1 Notation

ξ, d, m, σ Root seed, input width, number of components, and noise standard deviation.

x_t, g_t Current binary input and latent activity-gate states.

$y_t, \tilde{y}_t$ Noiseless and observed regression targets.

b_t, ϵ_t, σ Intercept, observation noise, and its standard deviation.

$f_i : \{0,1\}^d \to \{0,1\}, w_i$ Fixed binary indicator function and its fixed weight.

k_i, I_i Function arity and ordered selected-feature indices.

h_i Hamming threshold.

$p_{\text{table}}, k_{\text{table,max}}$ Truth-table family probability and maximum arity.

$k_{\text{threshold,min}}, k_{\text{threshold,max}}$ Hamming-threshold arity bounds.

$p_{\text{activation,min}}, p_{\text{activation,max}}$ Truth-table activation-probability range.

$q_{\max}, p_{\min}, p_{\max}$ Maximum parent count and CPT range.

$p_x, p^x_{\text{sample,min}}, p^x_{\text{sample,max}}$ Input distribution-sampling probability and node-sampling-probability range.

$p_g, p^g_{\text{sample,min}}, p^g_{\text{sample,max}}$ Gate distribution-sampling probability and node-sampling-probability range.

$w_{\min}, w_{\max}, b_{\min}, b_{\max}$ Weight and intercept ranges.

p_b Intercept-sampling probability.

S_t^x, S_t^g Input- and gate-distribution sampling indicators.

$S_{t,j}^x, S_{t,j}^g$ Input-node and gate-node sampling indicators.

S_t^b Intercept-sampling indicator.

2 Synthetic environment and target

At time t, an environment emits a binary vector $x_t \in \{0,1\}^d$ and an observed scalar target $\tilde{y}_t$. The environment also maintains binary activity gates $g_t \in \{0,1\}^m$, one per target component, which are never supplied to a model. Keeping g_i hidden keeps activity latent: if gates were observed, $g_i f_i(x)$ would collapse to just another visible component function, and the same f_i could not turn on and off without becoming a distinct observed feature. A latent target is

$$y_t = b_t + \sum_{i=1}^{m} w_i g_{t,i} f_i(x_t), \qquad \epsilon_t \sim \mathcal{N}(0, \sigma^2), \qquad \tilde{y}_t = y_t + \epsilon_t,$$

where ϵ_t is zero-mean Gaussian noise. This form asks models to discover structure in input bits. With enough boolean components it can in theory approximate arbitrary functions on binary inputs, while activity gates provide controllable levels of concept drift without rewriting the underlying components. Each $f_i : \{0,1\}^d \to \{0,1\}$ is a binary indicator (boolean component) represented by either a truth table or a Hamming threshold over a selected subset of input-bit indices. Truth tables and Hamming thresholds are simple, exact boolean generators with controllable arity and sparsity: tables stress arbitrary local interactions; thresholds stress monotone count structure. Together they cover more than one boolean style without a huge data-generating process. The selected-bit order is significant: the first bit is the most-significant bit in the truth-table index.

Fixed target construction. For each component, sample the truth-table family with probability p_{table}:

$$C_i \sim \text{Bernoulli}(p_{\text{table}}).$$

If $C_i = 1$, draw

$$\begin{aligned} k_i &\sim \text{DiscreteUniform}\{1, \ldots, k_{\text{table,max}}\}, \\ I_i &\sim \text{Uniform}\left(\binom{\{1, \ldots, d\}}{k_i}\right), \\ p_{\text{activation},i} &\sim \text{Uniform}(p_{\text{activation,min}}, p_{\text{activation,max}}), \end{aligned}$$

where $k_{\text{table,max}}$ is the maximum truth-table arity. Sample its 2^{k_i} lookup-table entries independently from Bernoulli($p_{\text{activation},i}$). The ordered selected bit values index the table. If $C_i = 0$, draw

$$\begin{aligned} k_i &\sim \text{DiscreteUniform}\{k_{\text{threshold,min}}, \ldots, k_{\text{threshold,max}}\}, \\ I_i &\sim \text{Uniform}\left(\binom{\{1, \ldots, d\}}{k_i}\right), \\ h_i &\sim \text{DiscreteUniform}\{1, \ldots, k_i\}, \end{aligned}$$

where $k_{\text{threshold,min}}$ and $k_{\text{threshold,max}}$ are the configured minimum and maximum Hamming-threshold arities. The component is

$$f_i(x) = \mathbf{1}\left\{\sum_{\ell=1}^{k_i} x_{I_{i,\ell}} \geq h_i\right\}.$$

Weights are sampled once, for example $w_i \sim \text{Uniform}(w_{\min}, w_{\max})$, and remain fixed for the trajectory. The initial intercept is

$$b_0 \sim \text{Uniform}(b_{\min}, b_{\max}).$$

Thus functions and weights are fixed after initialization. Holding components fixed keeps drift attributable to gates, intercept, noise, and input-state dynamics, and lets evaluation ask how fast a model re-adapts to scenarios it has already faced—for example because it built and kept reusable representations rather than forgetting them. Output changes arise from activity-gate transitions and independent intercept sampling.

3 Input and gate state dynamics

Let $X_t = (X_{t,1}, \ldots, X_{t,d})$ be the input state and $G_t = (G_{t,1}, \ldots, G_{t,m})$ the activity-gate state. We describe input sampling below. Gate sampling uses the same procedure independently.

$$X_t \sim P_x.$$

The input DAG has d nodes. At initialization, sample its topological order:

$$\pi_x \sim \text{Uniform}(\mathcal{S}_d).$$

For each node, uniformly sample a parent subset from all subsets of earlier nodes in its topological order with at most $q_{\max}$ parents. Independently sample every conditional Bernoulli probability in its CPT uniformly from $[p_{\min}, p_{\max}]$. Sample X_0 ancestrally from this DAG, then sample its shared node-sampling probability:

$$p^x_{\text{sample}} \sim \text{Uniform}\left(p^x_{\text{sample,min}}, p^x_{\text{sample,max}}\right).$$

Apply this construction independently to G_t: use m nodes, $\pi_g \sim \text{Uniform}(\mathcal{S}_m)$, and the gate-specific p_g, $p^g_{\text{sample,min}}$, and $p^g_{\text{sample,max}}$. Neither topological order alters the model-visible input-coordinate order.

State updates. After an emitted sample, draw the input-distribution sampling indicator:

$$S^x_t \sim \text{Bernoulli}(p_x).$$

If $S^x_t = 1$, sample a new input DAG order, parents, CPTs, and p^x_{sample} while retaining X_t. Then sample every input node through

$$S^x_{t,j} \sim \text{Bernoulli}(p^x_{\text{sample}}) \quad (j \in \{1, \ldots, d\}),$$

then, in topological order, ancestrally sample nodes with indicator one under the current input DAG.

Each sampled node is drawn from its CPT conditional on its current parent values. This partial ancestral sampling intentionally does not guarantee that the current state follows the configured DAG distribution. The gate state uses this same update independently: draw $S^g_t \sim \text{Bernoulli}(p_g)$, retain G_t if it is zero or sample a new gate distribution if it is one, then sample gate nodes in order with shared probability p^g_{sample}. Each shared node-sampling probability controls persistence and is sampled when its distribution is initialized or sampled again.

4 Stream-generation procedure

Configuration

Run Root seed ξ determines all random draws; d and m set the input width and number of target components; σ sets the observation-noise standard deviation.

Function p_{table}, $k_{\text{table,max}}$, $k_{\text{threshold,min}}$, $k_{\text{threshold,max}}$, $p_{\text{activation,min}}$, and $p_{\text{activation,max}}$ determine the function family, its arity bounds, and truth-table entry probabilities.

Distribution structure Parent limit $q_{\max}$ and CPT range $[p_{\min}, p_{\max}]$ bound each input and gate DAG's parents and conditional probabilities.

Input dynamics p_x controls new input-distribution sampling; $p^x_{\text{sample,min}}$ and $p^x_{\text{sample,max}}$ bound the shared input-node sampling probability.

Gate dynamics p_g controls new gate-distribution sampling; $p^g_{\text{sample,min}}$ and $p^g_{\text{sample,max}}$ bound the shared gate-node sampling probability.

Target scale $w_{\min}$, $w_{\max}$, $b_{\min}$, $b_{\max}$ bound weights and intercepts; p_b controls new intercept sampling.

Initialize: sample fixed functions f_i and weights w_i, initial intercept b_0, input and gate distributions, states X_0, G_0, and their shared node-sampling probabilities p^x_{sample} and p^g_{sample}.
For $t = 0, 1, \ldots$:

1. Set $o_t \leftarrow X_t$ and
$$\tilde{y}_t \leftarrow b_t + \sum_{i=1}^{m} w_i G_{t,i} f_i(X_t) + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, \sigma^2);$$
emit $(o_t, \tilde{y}_t)$.

2. Draw S^x_t and S^g_t. If either is one, resample the corresponding distribution; retain X_t or G_t.

3. For every input node j, draw $S^x_{t,j} \sim \text{Bernoulli}(p^x_{\text{sample}})$; in topological order, ancestrally sample nodes with indicator one to produce X_{t+1}. Repeat independently for every gate node using $S^g_{t,j} \sim \text{Bernoulli}(p^g_{\text{sample}})$ to produce G_{t+1}.

4. Draw $S^b_t \sim \text{Bernoulli}(p_b)$. If it is one, sample $b_{t+1} \sim \text{Uniform}(b_{\min}, b_{\max})$; otherwise set $b_{t+1} \leftarrow b_t$.

5 Drift mechanisms

Input and gate distribution sampling change their distributions abruptly. Neither sampling operation directly changes its family's current binary state, so latent activity continues from its pre-sampling value and is subsequently updated only by the usual partial ancestral sampling under the new distribution. Partial ancestral sampling provides persistent, within-family correlated states but does not necessarily follow either configured DAG distribution. Functions and weights do not drift. Activity-gate sampling and independent intercept sampling are the sources of output shifts. Gates remain unobserved activity variation. Oracle metadata is optional and is never supplied to a model.

6 API and integration boundaries

Swappable generators, models, and metrics are required for fair prequential benchmarks; coupling them would bake evaluation assumptions into model code. The `binaml.environments` package owns generation, configuration, serialization, and state restoration. `binaml.models` owns online models and has no environment dependency. A model implements `predict(features)` and `observe(features, target)`. The prediction call may retain its routing or matching context; `observe` receives the features and target, and may use the retained context or rerun inference. The `binaml.evaluation` package combines the two through predict-then-observe prequential evaluation. Named benchmark scenarios only compose these independent components.

7 Reference online baselines

The library provides two generic online baselines alongside the binary-feature regressor. The linear SGD baseline predicts with an affine function of the observed coordinates. The MLP baseline wraps scikit-learn's `MLPRegressor` with its `sgd` solver. It is optional and is installed through the `benchmarks` dependency extra so ordinary library users do not need scikit-learn.

All provided models use the same replay convention. After observing a target, they retain the most recent B labeled samples and make S optimization updates on that window. The linear baseline applies an averaged full-batch gradient update at each step. The MLP uses one `partial_fit` pass per step; its internal minibatch size equals the current replay-window length during warm-up. Reported comparisons must state each baseline's architecture, learning rate, regularization, replay size B, step count S, optimizer settings, and random seed.

8 Determinism and reproducibility

Every trajectory is determined by a validated JSON configuration and a single root seed ξ. The implementation splits that seed into stable PCG64DXSM substreams for initialization, sampling, drift, and noise. Snapshots include the full latent state and bit-generator states. Scenarios, generator versions, and configuration fingerprints are recorded with run artifacts.

9 Prequential evaluation protocol

For each emitted $(x_t, \tilde{y}_t)$, an evaluator first records the model prediction, then exposes the target through `observe`. Mean squared error is computed from the recorded pre-update predictions. Benchmarks publish their horizon, scenario configuration, seeds, generator version, and separate total, prediction, and observation timings; aggregate metrics alone are insufficient for replication.

Warm-up. A scenario may specify a non-negative `warmup_samples` count smaller than its horizon. Models predict and observe these initial samples normally, but their errors are excluded from reported MSE and RMSE. Time-series plots retain the full trajectory and mark the first evaluated sample with a vertical line so that the learning period remains visible.

10 Limitations and scope

The task is designed for controlled adaptation studies. It does not claim that synthetic drift matches a particular deployment, nor does this document make comparative performance claims. Results

may be added only for versioned scenarios with repeatable baseline runs.